

Infodemic Containment

Project walkthrough: what it does, how it was built, how the code works, and how to run it

Course	Foundations of AI (FOAI) case study
Topic	Misinformation spreaders vs. fact-checking moderators on a social network
Repository	github.com/adiseshan1505/FOAI-Case-Study
Code state	commit 3b6d897, 2026-09-21
Size	1,641 lines of source and experiment code, 337 lines of tests, across 31 Python and YAML files. 43 automated tests.

How to use this document. Section 1 explains the project in plain language and Section 2 shows how to run it. Section 3 is the full log of what was done from the first message to now, including the mistakes. Sections 4 and 5 explain how the code works, with the core logic annotated line by line. Sections 6 and 7 explain every output file and the results. Section 10 helps you present it.

Contents

1. What this project is	3
2. How to run it	4
3. What happened, from the first message to now	6
4. How the project is organized and how one run flows	9
5. Code walkthrough, line by line	12
6. The outputs, and how to read them	24
7. The results	27
8. The tests	29
9. Limitations and honest notes	29
10. Presenting the project	30
Appendix A. File inventory	32
Appendix B. Configuration reference	33

1. What this project is

This project is a simulation of a race. A **spreader** pushes false claims into a social network of 750 users, aiming at well-connected hubs and adapting when it gets caught. A **moderator** can check only a few claims per round against a small knowledge base, and has to decide what to do with claims it cannot verify, knowing that wrongly suppressing a true claim has a cost. The simulation is run thousands of times with different strategies on each side, and measures who wins, by how much, and under what conditions the moderator's strategy stops mattering.

The cast

Piece	What it is in the code
Spreader agent	Posts false claims each round for a limited campaign. Seeds them at random users or at hubs. If its claims get suppressed, it creates sockpuppet accounts that reshare its strongest claim.
Moderator agent	Has a budget B of checks per round. Ranks the queue of flagged claims, verifies the top B against the knowledge base, then quarantines, rate-limits, escalates or ignores.
Ordinary users	Not agents. 750 passive nodes in a scale-free graph. Each newly exposed user gets one chance to pass a claim to each neighbor (the Independent Cascade model).
Claims	Synthetic facts such as <code>capital_of(city_4, country_5)</code> . Each has a hidden true/false label that only the simulator knows.
Knowledge base	A small set of facts (a random 70% of the world) plus Horn-clause rules. Answers TRUE, FALSE or UNKNOWN.
Round	One time step: spreaders act, organic claims arrive, the moderator acts, then every claim spreads one hop. An episode runs 60 rounds.

The three questions it answers

ID	Question or hypothesis	How it is tested
RQ1 / H1	Does centrality-targeted spreading reach more users than random targeting?	Compare rows of the 2×3 matrix (C1 vs C4, C2 vs C5, C3 vs C6)
RQ1 / H2	Does impact-weighted moderation beat FIFO and random at equal budget?	Compare columns of the matrix (for example C4 vs C6)
RQ2 / H3	Does an adaptive moderator recover from the bot-swarm shock faster than a static one?	Apply the shock in all six conditions and compare recovery time
RQ3	How does λ trade exposure against false positives?	Sweep λ and plot the frontier

Key definitions used everywhere

- **Exposure.** A user counts as exposed once when a false claim first reaches them. The seed user of a false claim counts too. Sockpuppet accounts never count.
- **Infection curve.** The number of users newly exposed to false claims in each round. **Total exposure** is the area under it (the sum over rounds). **Peak infection** is its maximum.
- **False positive.** A true claim that was ever rate-limited or quarantined, counted once, even if it was later released.
- **Containment.** The first round after which new false exposures stay at or below 2 per round for the rest of the episode. If that never happens, time-to-containment is reported as the horizon (60).
- **Recovery time.** The same settling rule, measured from the shock round.

What it is not

- **Not built on a dataset.** The problem statement asks for synthetic predicates, a generated graph and a small hand-authored knowledge base. Nothing is downloaded or trained.
- **Not machine learning.** Both agents use explicit, inspectable rules, which keeps comparisons between strategies clean.

- **Not a model of a real platform.** Results describe the decision strategies, not real-world fact-checking accuracy.

2. How to run it

Everything is run from the repository root, F0AI-Case-Study/. It needs Python 3.10 or newer (developed on 3.14) and an internet connection for the first install.

Step by step

```
# 1. Create and activate a virtual environment (skip if .venv/ already exists)
python3 -m venv .venv
source .venv/bin/activate          # your prompt should now start with (.venv)

# 2. Install the project and its dependencies in editable mode
pip install -e ".[dev]"

# 3. Run the tests. Expect: 43 passed in about 2 seconds
pytest

# 4. Run the experiments. Each writes charts and tables into results/
python experiments/run_matrix.py      # H1, H2
python experiments/run_shock.py       # H3
python experiments/run_lambda_sweep.py # RQ3
python experiments/run_sensitivity.py  # robustness
```

Command	Answers	Measured time	Main outputs in results/
pytest	Does the code behave as designed?	2.3 s	none (console)
run_matrix.py	H1, H2: 6 conditions x 30 seeds, no shock	7.7 s	matrix_exposure.png, matrix_summary.csv, matrix_hypotheses.csv
run_shock.py	H3: same conditions with the bot-swarm shock	14.2 s	shock_recovery.png, shock_summary.csv, shock_hypotheses.csv
run_lambda_sweep.py	RQ3: exposure vs. false positives across λ	10.1 s	lambda_frontier.png, lambda_sweep.csv
run_sensitivity.py	Does H2 survive other budgets, KB coverage, sockpuppet caps?	about 82 s	three sensitivity_*.png charts, sensitivity_h2.csv

Try one simulation directly

```
python - <<'EOF'
from infodemic.config import load_config
from infodemic.env.simulation import run_simulation

cfg = load_config("experiments/configs/default.yaml")
result = run_simulation(cfg, "centrality", "impact_weighted", seed=0, shock=True)
print(result.metrics)          # total_exposure, peak_infection, time_to_containment, ...
print(result.stats[30])        # what happened in round 30, the shock round
EOF
```

The first argument is the spreader policy (random or centrality) and the second is the moderator policy (random, fifo, impact_weighted, plus the ablations impact_only and impact_static).

Command-line flags

Flag	Applies to	Effect
--seeds N	all runners	Use seeds 0 to N-1 instead of the 30 in the config. --seeds 5 gives a fast rough run. Pair it with --out, or the committed charts in results/ are overwritten.
--config PATH	all runners	Use another config file. Copy default.yaml, edit it, pass it here.
--out DIR	all runners	Write results somewhere other than results/. Use this for any experiment you do not want to keep.
--with-pure-impact	run_matrix.py	Add a ranking by velocity x reach alone, without the suspicion weighting.
--with-static	run_shock.py	Add an ablation whose impact ranking is frozen when a claim is first seen.
--k N	run_shock.py	Rounds allowed for recovery in the H3 check (default 15).
--condition C6, --lams 0,1,10	run_lambda_sweep.py	Which condition to sweep and which λ values.
--param NAME, --values 1,2,4	run_sensitivity.py	Sweep only one parameter, with your own values.

What you will see in the terminal

Each runner prints its tables as it finishes, then the paths of the files it wrote. This is the output of `run_matrix.py --with-pure-impact` with the default 30 seeds:

			total_exposure	peak_infection	time_to_containment	false_positives	false_positive_rate	
condition	spreader	moderator						
C1	random	random	466.300	45.500	33.200	0.433	0.011	
C2	random	fifo	506.633	44.233	35.367	0.300	0.007	
C3	random	impact_weighted	252.967	27.133	27.333	0.633	0.014	
C3p	random	impact_only	283.367	28.767	29.100	0.600	0.012	
C4	centrality	random	1073.167	83.233	33.500	0.633	0.015	
C5	centrality	fifo	1209.000	87.800	37.167	0.300	0.006	
C6	centrality	impact_weighted	442.233	40.233	26.600	1.167	0.023	
C6p	centrality	impact_only	442.767	40.800	31.667	1.067	0.020	
hypothesis				comparison	mean_diff	ci_low	ci_high	supported
H1		centrality - random targeting random		moderator	606.87	554.47	662.73	True
H1		centrality - random targeting fifo		moderator	702.37	643.06	762.20	True
H1		centrality - random targeting impact_weighted		moderator	189.27	151.70	228.67	True
H2		fifo - impact_weighted random		spreader	253.67	201.63	308.34	True
H2		random - impact_weighted random		spreader	213.33	167.53	266.00	True
H2		fifo - impact_only random		spreader	223.27	170.50	279.27	True
H2		fifo - impact_weighted centrality		spreader	766.77	706.83	825.37	True
H2		random - impact_weighted centrality		spreader	630.93	579.83	683.43	True
H2		fifo - impact_only centrality		spreader	766.23	699.20	829.87	True

If something goes wrong

Symptom	Cause and fix
ModuleNotFoundError: infodemic	The virtual environment is not active or step 2 was skipped. Run <code>source .venv/bin/activate</code> , then <code>pip install -e ".[dev]"</code> . As a fallback: <code>PYTHONPATH=src python experiments/run_matrix.py</code> .
pytest cannot find experiments/configs/	Run it from the repository root. One test loads the config files by relative path.
No module named pytest	Install with the dev extra: <code>pip install -e ".[dev]"</code> .
Different numbers from the document	The seeds are fixed, so the same code and config always give identical numbers. A different config, or a different --seeds, changes them.

3. What happened, from the first message to now

This is the complete log, in order. It includes the things that went wrong, because they explain why some parts of the project look the way they do.

Phase 1: the README

You attached the problem statement PDF and asked for a README.md and nothing else. I looked at the folder first (it held only the PDF and DOCX), then wrote a README that turns the problem statement into a project document.

- Sections: overview, the problem, the two agents, the environment, decision problems, objectives and the λ trade-off, the bot-swarm shock, research questions and hypotheses (RQ1 to RQ3, H1 to H3), the 2 x 3 evaluation matrix and metrics, scope and assumptions, references.
- The Vosoughi et al. citation came from your document. I added Kempe et al. (Independent Cascade) and Barabási and Albert (scale-free graphs) from memory and told you to check them.

Phase 2: getting it onto GitHub

You ran `git init`, committed, moved the problem statement into `docs/`, and added the remote. The push failed with "Repository not found". Debugging this took several rounds:

- I checked the remote (correct), the credential helper (store, with no saved file), and whether gh was installed (no).
- Using GitHub's public API I checked both candidate accounts. Both `adishesan1505` and `not-Jr` exist, and neither had a public repo of that name. You confirmed `adishesan1505` is your main account.
- The real clue was that git never asked for a password. Your terminal is VS Code's, which sets `GIT_ASKPASS` and silently supplies credentials from whichever GitHub account VS Code is signed into. That was almost certainly a different account.
- Bypassing it with `env -u GIT_ASKPASS` made git prompt, but GitHub rejected the password (it requires a token). Rather than debug tokens further, I gave you an SSH route, which you tried and it worked.
- Along the way you also changed your git author name to match, and amended the commit.

Phase 3: repository layout and tech stack

You asked for an initial file structure and then a tech stack section, both inside the README.

- I proposed a `src/infodemic/` package (`env`, `agents`, `policies`, `kb`, `scenarios`, `metrics.py`), plus `experiments/`, `tests/`, `results/` and `report/`. You asked for the comments to be removed from the tree, so I moved the explanations into a table underneath.
- Stack: Python, NetworkX (graph), NumPy (seeded randomness), pandas, Matplotlib, PyYAML, pytest, and a custom pure-Python Horn-clause engine so the inference stays transparent. I marked it as proposed because nothing existed yet.

Phase 4: the scaffold

You asked me to create the files from the README tree. I created every file as a stub: modules with only a docstring, three runners that raised `NotImplementedError`, two YAML configs (`default.yaml`, `conditions.yaml`), `requirements.txt`, `.gitignore`, and `.gitkeep` files so git would track the empty `results/` and `report/`. I compiled everything to confirm it was valid.

Phase 5: writing the code, file by file

You asked for the real code, file by file, with few comments. I wrote it in dependency order so each file could rely on the ones before it:

- `config.py`: typed config dataclasses, a YAML loader, validation, and an `override()` helper for parameter sweeps.
- `kb/`: the Horn-clause engine (`inference.py`), six hand-written rules (`rules.py`), and the synthetic world and fact generator (`facts.py`).

- `env/`: the scale-free network, claims with hidden labels, the Independent Cascade step, and the simulation loop.
- `policies/` and `agents/`: random and centrality seeding; random, FIFO and impact-weighted moderation; the spreader and moderator agents.
- `scenarios/bot_swarm.py` and `metrics.py`: the shock and the five metrics, plus a paired bootstrap for confidence intervals.

Phase 6: first run and calibration

The first smoke test ran one episode in 0.07 seconds. I then checked the knowledge base and tuned the settings.

- **Knowledge base soundness.** Over 2,000 true and 2,000 false claims, the KB gave zero wrong verdicts. It answered UNKNOWN for 866 of the true and 785 of the false claims (about 40%), which is the incompleteness we want.
- **Scarcity.** I set the moderator's budget to $B = 2$ and the spreader to 3 posts per round. The placeholder values in the stub config ($B = 5$, 2 posts) would have let every policy check everything and hidden the differences. The sensitivity analysis later confirmed this exactly.
- **λ frontier.** At $\lambda = 10$ false positives were very rare, so I swept λ from 0 to 200 to confirm the trade-off actually appears. It did, and it was monotone.

Phase 7: experiments and charts

I wrote the three experiment runners and a shared `common.py`, and loaded the data-visualization skill before writing any chart code.

- I validated the categorical palette with the skill's script. The three colors pass the color-blind separation checks. One (aqua) has low contrast on the light background, so the charts carry direct value labels and the CSV tables act as a text alternative.
- Color follows the entity: Random is always blue, FIFO orange, impact-weighted aqua, in every chart.
- I looked at the rendered charts instead of trusting the code. That caught overlapping labels on the λ chart (fixed by moving labels) and direct line labels on the shock chart that collided with the lines (removed in favor of an ordered legend).
- I had written a conclusion straight into a chart title. Results should decide that, not a script, so I replaced it with a neutral title.

Phase 8: tests and the README update

You asked whether the code was tested and how to run it.

- I wrote four test files (40 tests), created a project virtual environment, and installed the package in editable mode with `pip install -e ".[dev]"`.
- The first pytest run had one failure. It was my own arithmetic in a test (I expected a mean of 2.5; the differences 2, 5, 1 and 4 average 3.0), not a bug in the code.
- A lint pass with pyflakes found an unused import, which I removed. I also corrected two docstrings that no longer matched the code ("forward or backward chaining" and "publish counter-claims").
- I updated the README with usage, the modeling choices, the results and a Limitations section, and added a Quickstart at the top when you asked.

Phase 9: the audit before presenting

You asked whether the project was done. Before answering I tested the two things I had not tested: whether the spreader's adaptation matters, and whether my suspicion-weighting on the impact ranking drives H2. This found a real flaw.

- **The flaw.** When alarmed, the spreader stopped seeding at hubs and posted from sockpuppets instead. That helped the random spreader (+58 to +92 exposures) but hurt the centrality spreader (-112 to -178). A rational adversary would not adapt that way, so the centrality rows understated your strongest attacker.

- **The fix.** Sockpuppets now add amplification (resharing) while the spreader keeps seeding its own posts. Adapting now adds 56 to 114 exposures in every condition.
- **A wrong claim corrected.** I had told you H1 failed against the impact-weighted moderator (+2, CI -24 to 30). That was an artifact of the flaw. After the fix H1 holds against all three moderators (+189, CI 152 to 229). The earlier statement is superseded.
- **Ablation.** I added the `impact_only` policy (velocity x reach alone, exactly as the problem statement words it). H2 still holds, so it does not depend on my suspicion weighting.
- **Sensitivity analysis.** New `run_sensitivity.py`. Finding: H2 holds only while verification is scarce (B of 3 or less). It holds in 24 of 28 settings tested; all 4 failures are at B of 5 or 8.
- **False-positive floor.** At very high λ , about 0.27 false positives per run remained. I traced it to the 5% error rate of human review: with perfect review it drops to exactly 0.00.
- I added three tests (43 total), regenerated every result, rewrote the README numbers, and checked the Quickstart from a clean copy with a brand-new environment.

Phase 10: what gets pushed

You asked whether the results need to be pushed. I split `results/`: the six charts and the summary tables are tracked (the README embeds the charts, so they must be), and the six raw per-run CSVs are gitignored because they regenerate identically from the fixed seeds. I untracked the raw files with `git rm --cached` and updated the README's layout table.

You later added `.docs/` to `.gitignore` yourself. Note that this pattern matches a folder named `.docs`, not `docs/`, so it does not ignore your `docs/` folder.

Mistakes made, and how each was caught

Mistake	How it was caught	Fix
Spreader adaptation hurt the strongest attacker	Ad-hoc probe of adaptation on and off before presenting	Redesigned; new test asserts adaptation never hurts
Told you H1 failed against impact-weighted	Same probe; the result flipped after the fix	Regenerated all results, corrected the README
Conclusion hard-coded into a chart title	Own review of the runner code	Neutral title
Arithmetic error in a test (2.5 vs 3.0)	pytest failure	Corrected the expected value
Colliding chart labels	Looked at the rendered images	Moved or removed labels
Stale docstrings and one unused import	Re-reading the code; pyflakes	Corrected and removed

4. How the project is organized and how one run flows

4.1 The modules

Layer	Files	Job
Config	config.py, experiments/configs/*.yaml	Typed settings, YAML loading, validation, and override() for parameter sweeps
Knowledge	kb/facts.py, rules.py, inference.py	The synthetic world, the hand-written rules, and a backward-chaining engine that returns TRUE, FALSE or UNKNOWN
Environment	env/network.py, claims.py, cascade.py, simulation.py	The graph, claims with hidden labels, one Independent Cascade hop, and the round loop
Agents	agents/spreader.py, moderator.py	The two decision-makers
Policies	policies/spreader_policies.py, moderator_policies.py	Swappable strategies: random and centrality seeding; random, FIFO and impact-weighted claim selection
Scenario	scenarios/bot_swarm.py	The coordinated shock
Metrics	metrics.py	Per-round stats, the five run metrics, and the paired bootstrap
Experiments	experiments/common.py and four run_*.py	Run every condition over many seeds, write tables, draw charts
Tests	tests/ (4 files)	43 automated checks

4.2 What happens in one episode

A single call `run_simulation(cfg, spreader, moderator, seed, shock)` does the following:

1. **Build the world.** `Simulation.__init__` creates the scale-free graph, the synthetic world and the knowledge base (a random 70% of the world's facts), then the two agents and, if `shock=True`, the bot swarm.
2. **Loop for 60 rounds.** Each call to `step()` runs the round described below.
3. **Compute metrics.** `compute_metrics()` turns the 60 per-round records into total exposure, peak, time-to-containment, false-positive rate and recovery time.

One round, in order

#	Step	What happens
1	Spreader acts	For rounds 0 to 19 it posts 3 false claims, seeded by the chosen policy. If one of its claims was suppressed in the last 3 rounds it also deploys a sockpuppet and has its sockpuppets reshare its strongest live claim.
2	Swarm acts (shock runs only)	At the shock round, 12 extra spreaders each post 3 claims per round for 3 rounds.
3	Organic claims arrive	A random number of claims (mean 1) from random users, each false with probability 0.1.
4	Moderator acts	Claims flagged one round after creation join the queue. Human reviews that are due are resolved. The policy picks up to $B = 2$ claims. Each is checked against the knowledge base and acted on.
5	Cascade	Every claim spreads one Independent Cascade hop, unless quarantined. Rate-limited claims spread at 30% probability.
6	Record	New false exposures, active false claims, queue length, takedowns and false positives are stored for this round.

4.3 The key data structures

The most important design idea is the split between what the simulator knows and what the moderator can see.

Claim (simulator only)	ClaimView (what the moderator sees)
id, atom, source, created_round, content_score, origin, status	id, atom, source, created_round, content_score, status
is_false: the hidden ground-truth label	Absent. A test asserts the field does not exist.
exposed (set of users), frontier (users newly exposed, about to pass it on), blocked (users stopped by quarantine)	velocity = number of users in the frontier, reach = summed degree of the frontier, expected_next = expected new exposures next hop, exposed_count
takedown_counted: has this claim already counted as a takedown?	p_false: the moderator's own estimate that the claim is false

A claim's status is a small state machine: ACTIVE can become RATE_LIMITED or QUARANTINED; RATE_LIMITED can go back to ACTIVE (if human review says the claim is true) or on to QUARANTINED; QUARANTINED is final. **A user's state, per claim**, is derived: not in exposed means clean, in exposed means exposed, in blocked means quarantined.

4.4 Randomness and paired comparisons

The simulation has eight independent random streams: graph, world, organic, spreader, moderator, cascade, swarm, review. They are spawned from the run's seed. As a result, one seed gives the same graph, the same knowledge base and the same organic claims in every condition, whichever strategies are being compared. This lets every comparison be **paired by seed**, which makes the confidence intervals much tighter than comparing independent runs.

4.5 Why the design looks this way

Decision	Reason
Backward chaining, with negative knowledge written as not_<predicate> rules	Verification is goal-driven, so a rule's head variables are already bound when it runs. It also keeps the engine within pure Horn clauses (no negation as failure).
The infection curve counts new exposures per round	Exposure cannot be undone, so quarantine cannot lower a running total. What a moderator can change is how many new exposures follow.
The knowledge base is sound but incomplete	It never gives a wrong verdict, so false positives can come only from acting on claims it could not resolve. This is where λ acts.
λ acts only on UNKNOWN claims	A verified verdict needs no trade-off. The provisional rate-limit is where suppressing a true claim is a real risk.
Sockpuppets are extra graph nodes that are never counted as users	They amplify a claim without inflating the exposure count.
The main campaign ends at round 20 and the shock starts at round 30	Containment can be reached first, so recovery is measured against an absolute threshold.
Impact is multiplied by the estimated probability a claim is false	Avoids spending checks on likely-true claims. The impact_only ablation shows the result does not depend on it.

4.6 A real worked example

This is genuine output, not an illustration. It is the first six rounds of one run: centrality-informed spreader against the impact-weighted moderator, seed 0, default config. "Cascade" counts every user newly exposed to a false claim that round, including the seeds of the spreader's fresh posts.

```

--- round 0 ---
post claim 0 [spreader] FALSE located_in(city_12, region_2) seeded at node 66 (degree 17)
post claim 1 [spreader] FALSE capital_of(city_4, country_5) seeded at node 18 (degree 20)
post claim 2 [spreader] FALSE allied(country_10, country_37) seeded at node 22 (degree 17)
post claim 3 [organic ] true  allied(country_20, country_5) seeded at node 180 (degree 10)
moderator queue size 0, budget B=2
cascade: 4 users newly exposed to false claims; takedowns so far 0, false positives 0
--- round 1 ---
post claim 4 [spreader] FALSE located_in(city_24, region_2) seeded at node 34 (degree 17)
post claim 5 [spreader] FALSE capital_of(city_0, country_11) seeded at node 37 (degree 20)
post claim 6 [spreader] FALSE located_in(city_18, region_2) seeded at node 8 (degree 56)
post claim 7 [organic ] true  capital_of(city_25, country_25) seeded at node 537 (degree 6)
post claim 8 [organic ] true  allied(country_29, country_0) seeded at node 271 (degree 4)
moderator queue size 4, budget B=2
check claim 1: p_false=0.86 velocity=1 reach=39 impact=39 -> KB says FALSE (truth: FALSE); active -> quarantined
check claim 3: p_false=0.53 velocity=1 reach=10 impact=10 -> KB says TRUE (truth: true); active -> active
cascade: 18 users newly exposed to false claims; takedowns so far 1, false positives 0
--- round 2 ---
post claim 9 [spreader] FALSE capital_of(city_8, country_32) seeded at node 26 (degree 21)
post claim 10 [spreader] FALSE allied(country_3, country_37) seeded at node 77 (degree 21)
post claim 11 [spreader] FALSE allied(country_20, country_29) seeded at node 8 (degree 56)
spreader alarmed -> deployed 1 sockpuppet(s); total 1
moderator queue size 7, budget B=2
check claim 6: p_false=0.89 velocity=12 reach=163 impact=1956 -> KB says FALSE (truth: FALSE); active -> quarantined
check claim 8: p_false=0.48 velocity=3 reach=36 impact=108 -> KB says TRUE (truth: true); active -> active
cascade: 13 users newly exposed to false claims; takedowns so far 2, false positives 0

--- round 3 ---
post claim 12 [spreader] FALSE allied(country_17, country_27) seeded at node 33 (degree 25)
post claim 13 [spreader] FALSE capital_of(city_14, country_10) seeded at node 11 (degree 36)
post claim 14 [spreader] FALSE capital_of(city_31, country_25) seeded at node 24 (degree 32)
post claim 15 [organic ] true  allied(country_0, country_18) seeded at node 218 (degree 6)
spreader alarmed -> deployed 1 sockpuppet(s); total 2
moderator queue size 8, budget B=2
check claim 11: p_false=0.83 velocity=7 reach=82 impact=574 -> KB says FALSE (truth: FALSE); active -> quarantined
check claim 9: p_false=0.83 velocity=2 reach=13 impact=26 -> KB says FALSE (truth: FALSE); active -> quarantined
cascade: 14 users newly exposed to false claims; takedowns so far 4, false positives 0
--- round 4 ---
post claim 16 [spreader] FALSE allied(country_5, country_4) seeded at node 83 (degree 18)
post claim 17 [spreader] FALSE located_in(city_36, region_1) seeded at node 38 (degree 19)
post claim 18 [spreader] FALSE capital_of(city_31, country_15) seeded at node 83 (degree 18)
post claim 19 [organic ] true  allied(country_39, country_13) seeded at node 555 (degree 3)
post claim 20 [organic ] true  capital_of(city_12, country_12) seeded at node 96 (degree 8)
post claim 21 [organic ] true  allied(country_24, country_4) seeded at node 16 (degree 15)
spreader alarmed -> deployed 1 sockpuppet(s); total 3
moderator queue size 10, budget B=2
check claim 13: p_false=0.58 velocity=9 reach=55 impact=495 -> KB says FALSE (truth: FALSE); active -> quarantined
check claim 12: p_false=0.91 velocity=2 reach=10 impact=20 -> KB says UNKNOWN (truth: FALSE); active -> rate_limited
cascade: 9 users newly exposed to false claims; takedowns so far 6, false positives 0
--- round 5 ---
post claim 22 [spreader] FALSE located_in(city_32, region_2) seeded at node 35 (degree 31)
post claim 23 [spreader] FALSE capital_of(city_21, country_2) seeded at node 34 (degree 18)
post claim 24 [spreader] FALSE capital_of(city_15, country_20) seeded at node 66 (degree 17)
spreader alarmed -> deployed 1 sockpuppet(s); total 4
moderator queue size 14, budget B=2
check claim 10: p_false=0.94 velocity=6 reach=65 impact=390 -> KB says FALSE (truth: FALSE); active -> quarantined
check claim 18: p_false=0.85 velocity=3 reach=65 impact=195 -> KB says FALSE (truth: FALSE); active -> quarantined
cascade: 7 users newly exposed to false claims; takedowns so far 8, false positives 0

```

What to notice

- **Round 0 has an empty queue.** Claims are flagged one round after they are posted (the detection delay), so the moderator has nothing to check yet.
- **Round 1: impact decides.** Four claims are queued and $B = 2$. Claim 1 (impact 39, p_false 0.86) is checked first and quarantined. The organic claim 3 is second and turns out true, so one check was spent on a true claim. That is the cost of ranking on imperfect evidence.
- **Round 2: a hub claim has grown.** Claim 6 was seeded at a degree-56 hub. It now has velocity 12 and reach 163, so its impact is 1,956, far above anything else. It is checked first and quarantined. Also in round 2, the spreader saw its claim 1 suppressed in round 1, so it became alarmed and deployed its first sockpuppet.
- **Rounds 3 to 5: the sockpuppet count grows** by one each round (2, 3, then 4) while the spreader keeps being suppressed.
- **Round 4: the unknown-claim rule at work.** For claim 12 the knowledge base answers UNKNOWN, but the moderator estimates $p_false = 0.91$. With that much suspicion the cost side of the rule, $\lambda \times (1 - p_false)$, is small, so the claim is rate-limited provisionally. The simulator's truth column shows the claim really was false.
- **Queue growth.** The queue goes 0, 4, 7, 8, 10, 14 because 3 to 6 claims arrive each round and only 2 are checked. That imbalance is exactly why prioritization matters.

5. Code walkthrough, line by line

The source and experiment code is about 1,600 lines, so annotating every line of every file would bury the important ones. This section annotates **every line of the core logic** (the parts that make the simulation behave the way it does): config overrides, the inference engine, the network, the cascade, both agents, the moderator's selection policy, the simulation loop, the shock and the metrics. Line numbers are those in the source files. The remaining files (dataclass definitions, the YAML loader, chart styling, the CLI plumbing) are described in prose.

How to read the tables. Left: the exact source lines with their line numbers. Right: what those lines do and why. Uppercase names in Horn rules (like X or C) are logic variables, not constants.

5.1 Configuration: config.py

All tunable numbers live in frozen dataclasses (GraphConfig, SpreaderConfig, ModeratorConfig, ClaimConfig, ShockConfig, SimConfig). `load_config()` reads a YAML file into them, rejecting unknown keys. `validate()` checks ranges (for example, a negative λ or a shock that runs past the horizon raises `ValueError`). The one function worth reading closely is `override()`, because every parameter sweep uses it.

`override()`: change one setting without mutating anything

src/infodemic/config.py, starting at line 125

125 def override(cfg: SimConfig, updates: dict[str, object]) -> SimConfig:	Takes a config and a dict of dotted paths to new values, such as {"moderator.lam": 5}.
126 """Return a copy of cfg with dotted-path updates applied, e.g. {"moderator.lam": 5}."""	The docstring shows the format.
128 def set_path(obj, parts, value):	Helper that changes one value deep inside nested configs, one level per call.
129 head, *rest = parts	Splits ["moderator", "lam"] into head = "moderator" and rest = ["lam"].
130 new = set_path(getattr(obj, head), rest, value) if rest else value	If there is more path left, recurse into the child object to build its updated copy. Otherwise this is the leaf, so use the new value.
131 return dataclasses.replace(obj, **{head: new})	The configs are frozen (immutable), so <code>dataclasses.replace</code> builds a new copy with just that one field changed.
133 for path, value in updates.items():	Apply every requested update in turn.
134 cfg = set_path(cfg, path.split("."), value)	Each update rebuilds the config from the top.
135 return validate(cfg)	Validate the result, so a bad sweep value (like a negative λ) fails immediately instead of silently producing nonsense.

5.2 The knowledge base: kb/

The knowledge base is the Horn-clause layer from the problem statement. A **fact** is a tuple such as ("capital_of", "city_4", "country_4"). A **rule** is (head, body): the head is true if every atom in the body can be proven. The world (facts.py) gives every country one capital, one region and one bloc. The moderator's KB receives a random 70% of those base facts, plus these six rules.

rules.py: the six hand-written rules

src/infodemic/kb/rules.py, starting at line 6

6 RULES: list[Rule] = [7 (("located_in", "X", "R"), (("capital_of", "X", "C"), ("part_of", "C", "R"))), 8 (("allied", "A", "B"), (("member_of", "A", "G"), ("member_of", "B", "G"), ("neq", "A", "B"))), 9 (("not_capital_of", "X", "C"), (("capital_of", "X", "D"), ("neq", "C", "D"))), 10 (("not_capital_of", "X", "C"), (("capital_of", "Y", "C"), ("neq", "X", "Y"))), 11 (12 ("not_located_in", "X", "R"), 13 (("capital_of", "X", "C"), ("part_of", "C", "S"), ("neq", "R", "S")), 14), 15 (16 ("not_allied", "A", "B"), 17 (("member_of", "A", "G"), ("member_of", "B", "H"), ("neq", "G", "H")), 18), 19]	The rule list. Uppercase names are variables that the engine binds while proving. A city is located in a region if it is the capital of some country C and that country is part of the region. This lets the KB prove <code>located_in</code> claims it was never directly told. Two countries are allied if both are members of the same bloc G and they are different countries. <code>neq</code> is a built-in inequality test. Negative knowledge, form 1. X is <i>*not*</i> the capital of C if X is known to be the capital of some other country D. Negative knowledge, form 2. ...or if C is known to have a different capital Y. X is <i>*not*</i> located in region R if it is the capital of C, C is in region S, and S differs from R. A and B are <i>*not*</i> allied if they belong to different blocs G and H. End of the list.
--	--

The engine that uses those rules is in `inference.py`. First the three small helpers that implement **unification**, the matching step at the heart of logic programming:

is_var, walk, unify: the matching machinery

src/infodemic/kb/inference.py, starting at line 21

21 def is_var(term: str) -> bool: 22 return term[:1].isupper()	A term is a variable if it starts with an uppercase letter. Constants are lowercase.
25 def walk(term: str, subst: Subst) -> str: 26 while is_var(term) and term in subst: 27 term = subst[term] 28 return term	<code>walk</code> follows a chain of bindings to its end. If X is bound to Y and Y to <code>city_4</code> , walking X returns <code>city_4</code> .
31 def unify(a: Atom, b: Atom, subst: Subst) -> Subst None: 32 if len(a) != len(b): 33 return None	<code>unify</code> tries to make two atoms identical by choosing variable bindings. Atoms of different length can never match, so return None (failure).
34 for x, y in zip(a, b): 35 x, y = walk(x, subst), walk(y, subst)	Compare the atoms position by position, first resolving each side through the current bindings.
36 if x == y: 37 continue	Identical terms need no new binding; move on.
38 if is_var(x): 39 subst = {**subst, x: y}	If the left term is an unbound variable, bind it to the right term. (<code>{**subst, x: y}</code> copies the dict, so earlier states are not corrupted.)
40 elif is_var(y): 41 subst = {**subst, y: x}	Otherwise, if the right term is the variable, bind it to the left.
42 else: 43 return None	Two different constants cannot match, so the whole unification fails.
44 return subst	Every position matched: return the extended bindings.

`verify()` is the public entry point the moderator calls. It turns the engine's yes/no answers into a three-valued verdict:

verify(): three-valued verdicts

src/infodemic/kb/inference.py, starting at line 69

69	def verify(self, atom: Atom) -> Verdict:	Given a claim atom, return TRUE, FALSE or UNKNOWN.
70	if atom not in self._cache:	Verdicts are cached. The KB never changes during a run, so each distinct claim is proven only once.
71	if self.holds(atom):	First try to prove the claim itself.
72	verdict = Verdict.TRUE	
73	elif self.holds(("not_" + atom[0], *atom[1:])):	If that fails, try to prove its negation : the same atom with not_ in front of the predicate name (not_capital_of, and so on).
74	verdict = Verdict.FALSE	
75	else:	If neither can be proven, the knowledge base is silent. This is the case that forces the moderator to choose between escalating, rate-limiting and ignoring.
76	verdict = Verdict.UNKNOWN	
77	self._cache[atom] = verdict	Store the verdict.
78	return self._cache[atom]	Return it.

_solve is the proof search itself (backward chaining, also called SLD resolution). It is a generator: it yields every set of variable bindings that proves all the goals.

_solve(): backward-chaining proof search

src/infodemic/kb/inference.py, starting at line 89

89	def _solve(self, goals: list[Atom], subst: Subst, depth: int) -> Iterator[Subst]:	Prove the list of goals, given the bindings so far. depth counts how deeply rules are nested.
90	if not goals:	No goals left means every goal has been proven. Yield the bindings and stop.
91	yield subst	
92	return	
93	goal, rest = goals[0], goals[1:]	Work on the first goal; the others wait in rest.
94	if goal[0] == "neq":	Built-in neq. It succeeds only if both terms are already bound to constants and differ. If a term is still a variable it fails, which is why each rule must bind its variables in earlier body atoms.
95	a, b = walk(goal[1], subst), walk(goal[2], subst)	
96	if not is_var(a) and not is_var(b) and a != b:	
97	yield from self._solve(rest, subst, depth)	
98	return	
99	for fact in self._facts.get(goal[0], ()):	Facts. Try every stored fact with the same predicate. If it unifies with the goal, keep going with the remaining goals under the extended bindings.
100	extended = unify(goal, fact, subst)	
101	if extended is not None:	
102	yield from self._solve(rest, extended, depth)	
103	if depth >= MAX_DEPTH:	Stop expanding rules beyond depth 8, so a badly written recursive rule cannot loop forever. A test checks this.
104	return	
105	for rule in self._rules.get(goal[0], ()):	Rules. For each rule whose head has this predicate: rename its variables to fresh ones (so separate uses cannot clash), unify the goal with the head, and replace the goal with the rule's body, which is a smaller problem. Then continue.
106	head, body = self._rename(rule)	
107	extended = unify(goal, head, subst)	
108	if extended is not None:	
109	yield from self._solve([*body, *rest], extended, depth + 1)	

5.3 The network: env/network.py

Network wraps a NetworkX graph. Nodes 0 to 749 are real users; any node numbered higher is a sockpuppet (is_bot). Its four key lines:

build_network(): a scale-free graph with trust scores

src/infodemic/env/network.py, starting at line 51

51 def build_network(cfg: GraphConfig, rng: np.random.Generator) -> Network:	Builds the network for one episode from the graph config and the graph random stream.
52 graph = nx.barabasi_albert_graph(cfg.num_nodes, cfg.attachm ent_edges, seed=int(rng.integers(2**31)))	A Barabási-Albert graph: each new node attaches to 3 existing nodes, preferring ones that already have many links. That produces the heavy-tailed degree distribution with a few very influential hubs.
53 trust = rng.uniform(*cfg.trust_range, size=cfg.num_nodes)	Each user gets a trust score drawn uniformly from 0.6 to 1.4.
54 return Network(graph, trust, cfg.base_reshare_prob, cfg.soc kuppet_trust)	Bundle the graph, trust scores, base reshare probability (0.1) and sockpuppet trust.

edge_prob()

src/infodemic/env/network.py, starting at line 37

37 def edge_prob(self, u: int, v: int) -> float:	The probability that user u passes a claim to neighbor v.
38 return min(1.0, self._base_prob * self._trust[u])	Base probability 0.1 times the sender's trust score, capped at 1. Trusted users spread claims further.

hubs()

src/infodemic/env/network.py, starting at line 40

40 def hubs(self, fraction: float) -> list[int]:	The hubs are the top fraction of users by degree (5% by default).
41 return self._hub_order[: max(1, int(fraction * self.num _humans))]	_hub_order was sorted once at build time. Slice off the front, keeping at least one.

add_sockpuppet()

src/infodemic/env/network.py, starting at line 43

43 def add_sockpuppet(self, targets: Iterable[int]) -> int:	Create a sockpuppet account linked to the given target users.
44 node = self.graph.number_of_nodes()	The new node's id is the next free number, always above 749.
45 self.graph.add_node(node)	Add it to the graph.
46 self.graph.add_edges_from((node, t) for t in targets)	Link it to each target, so a claim it carries can reach them.
47 self._trust.append(self._sockpuppet_trust)	Give it a trust score.
48 return node	Return its id. It carries claims but never counts as an exposed user.

5.4 Claims and what the moderator sees: env/claims.py

Claim.view() is the boundary of partial observability. It converts the simulator's full Claim into the ClaimView the moderator is allowed to see, and it is where velocity, reach and the expected next-hop exposure are computed.

Claim.view(): the boundary of partial observability

src/infodemic/env/claims.py, starting at line 75

75	def view(self, net: Network, rate_limit_factor: float) -> ClaimView:	Build the moderator's view of this claim.
76	factor = self.spread_factor(rate_limit_factor)	The spread factor: 1.0 normally, 0.3 if rate-limited, 0 if quarantined.
77	expected = sum(78 net.edge_prob(u, v) * factor 79 for u in self.frontier 80 for v in net.neighbors(u) 81 if v not in self.exposed and not net.is_bot(v) 82)	Expected new exposures next hop. For each carrier u in the frontier, and each neighbor v that is not yet exposed and is a real user, add the reshare probability times the factor.
83	return ClaimView(84 id=self.id, 85 atom=self.atom, 86 source=self.source, 87 created_round=self.created_round, 88 content_score=self.content_score, 89 status=self.status,	Plain fields are copied across. is_false is deliberately not one of them.
90	velocity=len(self.frontier),	Velocity: how many users were exposed in the last hop.
91	reach=sum(net.degree(u) for u in self.frontier),	Reach: the total number of neighbors those users have, meaning how far the claim could go next.
92	expected_next=expected,	The expected next-hop exposure computed above. The moderator's λ rule uses this.
93	exposed_count=len(self.exposed), 94)	How many users have seen it in total, then the closing bracket.

5.5 The spread of a claim: env/cascade.py

This is the Independent Cascade model in about a dozen lines. It runs once per claim per round.

cascade_step(): one Independent Cascade hop

src/infodemic/env/cascade.py, starting at line 16

16	factor = claim.spread_factor(rate_limit_factor)	1.0, 0.3 or 0, depending on whether the claim is active, rate-limited or quarantined.
17	if factor == 0.0 or not claim.frontier: 18 claim.frontier = [] 19 return 0	A quarantined claim, or one with nobody left to pass it on, does not spread. Clear the frontier and report zero new exposures.
20	newly_exposed: list[int] = []	Collect the users newly exposed this hop.
21	for u in claim.frontier:	Each carrier in the frontier (the users exposed in the previous hop)...
22	for v in net.neighbors(u):	...gets one chance at each of its neighbors.
23	if v in claim.exposed or net.is_bot(v): 24 continue	Skip neighbors that have already seen this claim (nobody is exposed twice) and sockpuppets (they are not counted as users).
25	if rng.random() < net.edge_prob(u, v) * factor: 26 claim.exposed.add(v) 27 newly_exposed.append(v)	The random draw. With probability $\text{edge_prob} \times \text{factor}$, the neighbor is exposed. Record it in exposed and in the new-exposure list.
28	claim.frontier = newly_exposed	The newly exposed users become the next frontier: they get their own single chance next round.
29	return len(newly_exposed)	Return how many there were. The simulation adds this to the infection curve for false claims.

5.6 The spreader: agents/spreader.py

The spreader seeds three false claims per round for the first 20 rounds. It sees only one thing about the moderator: the public log of which claims were suppressed. If any of its own claims were suppressed in the last 3 rounds, it becomes alarmed and escalates by adding sockpuppets. The seeding policy (policies/spreader_policies.py) is separate: RandomSeeding picks uniformly among all users, and CentralitySeeding picks uniformly among the top 5% of users by degree.

SpreaderAgent.act(): one round of the spreader

src/infodemic/agents/spreader.py, starting at line 45

45	def act(self, sim: Simulation, r: int) -> None:	Called once per round with the simulation and the round number.
46 47	if not self.start <= r < self.end: return	Only active inside its window: rounds 0 to 19 for the main spreader. The shock's spreaders get their own windows.
48	alarmed = self.adaptive and self._alarmed(sim, r)	Is it alarmed? Only adaptive spreaders check. Swarm spreaders are not adaptive.
49 50	if alarmed: self._deploy_sockpuppets(sim)	If so, add sockpuppets first, so they can be used this round.
51 52 53	for _ in range(self.cfg.posts_per_round): source = self.policy.pick(sim.net, 1, self.rng)[0] self.claim_ids.append(sim.post_claim(source, True, "spreader", self.rng).id)	Post posts_per_round false claims. The seeding policy chooses each source (a random user or a hub). post_claim creates the claim, seeds it, and returns it, and the spreader remembers its id.
54 55	if alarmed: self._reshare(sim)	When alarmed, also have the sockpuppets reshare its strongest live claim.

_alarmed()

src/infodemic/agents/spreader.py, starting at line 57

57	def _alarmed(self, sim: Simulation, r: int) -> bool:	Has the moderator been visibly acting against this spreader?
58	return bool(sim.recent_suppressions(r - ALARM_MEMORY) & set(self.claim_ids))	True if any of its own claim ids (set intersection) appears among suppressions in the last 3 rounds (ALARM_MEMORY). The log says only which claims were suppressed, not why.

_deploy_sockpuppets()

src/infodemic/agents/spreader.py, starting at line 60

60	def _deploy_sockpuppets(self, sim: Simulation) -> None:	Create new sockpuppet accounts.
61	room = self.cfg.max_sockpuppets - len(self.sockpuppets)	How many more it is allowed: the cap (10) minus what it already has.
62 63	for _ in range(min(self.cfg.sockpuppets_per_round, room)): links = self.policy.pick(sim.net, self.cfg.sockpupp et_links, self.rng)	At most one per round (sockpuppets_per_round), and never beyond the cap.
64	self.sockpuppets.append(sim.net.add_sockpuppet(link s))	Choose 8 accounts to link to, using the same targeting policy: hubs for the centrality spreader, random users otherwise.
		Create the account in the graph and keep track of it.

`_reshare()`

src/infodemic/agents/spreader.py, starting at line 66

66	def _reshare(self, sim: Simulation) -> None:	Have sockpuppets amplify the spreader's best claim.
67	views = [sim.view(cid) for cid in self.claim_ids]	Look up the view of each of its own claims.
68	live = [v for v in views if v.status is not ClaimStatus.QUARANTINED]	Keep those that are not quarantined.
69	if not live:	Nothing live to boost: do nothing.
70	return	
71	target = max(live, key=lambda v: v.exposed_count)	Pick the claim that has reached the most users so far.
72	for node in self.sockpuppets:	Each sockpuppet adds itself as a carrier of that claim, giving it another chance to pass the claim to the hubs it is linked to. The spreader's own seeding is untouched, which is why adapting can never cost it.
73	sim.reshare(target.id, node)	

5.7 Ranking claims: policies/moderator_policies.py

RandomModeration picks budget claims at random. FIFOModeration sorts by arrival, oldest first. The interesting one is ImpactWeightedModeration, which the problem statement calls impact-weighted moderation.

ImpactWeightedModeration: the adaptive ranking

src/infodemic/policies/moderator_policies.py, starting at line 35

35	def __init__(self, refresh: bool = True, use_suspicion: bool = True):	Two switches. refresh recomputes the score each round (adaptive) instead of freezing it. use_suspicion multiplies by the estimated chance the claim is false. _frozen remembers scores for the static ablation.
36	self.refresh = refresh	
37	self.use_suspicion = use_suspicion	
38	self._frozen: dict[int, float] = {}	
40	def _score(self, view: ClaimView) -> float:	Score = impact , which is velocity x reach, optionally times p_false.
41	return view.impact * (view.p_false if self.use_suspicion else 1.0)	
43	def select(self, queue: Sequence[ClaimView], budget: int, rng: np.random.Generator) -> list[ClaimView]:	Choose up to budget claims from the queue.
44	scores = {}	Scores for this round.
45	for view in queue:	For each queued claim: if adaptive, compute a fresh score from its current velocity and reach, so the ranking follows the spread. In the static ablation, remember the score from the first time the claim was seen and reuse it forever.
46	if self.refresh:	
47	scores[view.id] = self._score(view)	
48	else:	
49	scores[view.id] = self._frozen.setdefault(view.id, self._score(view))	
50	tie_break = (lambda v: -v.p_false) if self.use_suspicion else (lambda v: 0.0)	Ties are common (every claim that stopped spreading scores 0). Break them by higher p_false, or ignore suspicion in the impact_only ablation.
51	ranked = sorted(queue, key=lambda v: (-scores[v.id], tie_break(v), v.id))	Sort by score, highest first (then tie-break, then id, so results are deterministic) and take the first budget.
52	return ranked[:budget]	

5.8 The moderator: agents/moderator.py

The moderator's round has four steps: flag, estimate, select and verify, act. It never touches Claim.is_false. The only way it learns the truth is a KB verdict or a human review.

Moderator.act(): one round of the moderator

src/infodemic/agents/moderator.py, starting at line 41

41	def act(self, sim: Simulation, r: int) -> None:	Called once per round.
42	self.queue.extend(sim.claims_created_at(r - self.cfg.detection_delay))	Claims created detection_delay (1) rounds ago are flagged now and join the queue.
43	self._resolve_reviews(sim, r)	First settle any human reviews that have come due.
44	views = [self._annotate(sim.view(cid)) for cid in self.queue]	Build the moderator's view of every queued claim, add its own estimate p_false, and drop claims older than 25 rounds (queue_ttl).
45	views = [v for v in views if r - v.created_round <= self.cfg.queue_ttl]	
46	self.queue = [v.id for v in views]	Keep the queue in sync with what survived.
47	for view in self.policy.select(views, self.cfg.budget_per_round, self.rng):	The policy picks up to B claims; each one is verified and acted on.
48	self._verify(sim, view, r)	

_annotate(): from evidence to a probability

src/infodemic/agents/moderator.py, starting at line 50

50	def _annotate(self, view: ClaimView) -> ClaimView:	Attach the moderator's estimate that a claim is false.
51	bad, good = self.ledger.get(view.source, (0, 0))	The credibility ledger: how many of this source's past claims were verified false (bad) and true (good).
52	suspicion = (bad + 1) / (bad + good + 2)	A smoothed suspicion rate. An unknown source scores 0.5.
53	z = CONTENT_WEIGHT * (view.content_score - CONTENT_MIDPOINT) + SOURCE_WEIGHT * (suspicion - 0.5)	Log-odds of falsehood: 8 x (content score - 0.525) + 3 x (suspicion - 0.5). The midpoint 0.525 sits between typical content scores of false claims (0.65) and true claims (0.40).
54	return replace(view, p_false=1.0 / (1.0 + math.exp(-z)))	Convert to a probability with the logistic function and return a new view. replace copies the frozen view.

_verify()

src/infodemic/agents/moderator.py, starting at line 59

59	def _verify(self, sim: Simulation, view: ClaimView, r: int) -> None:	Check one claim.
60	self.queue.remove(view.id)	It leaves the queue, whatever the outcome.
61	verdict = self.kb.verify(view.atom)	Ask the knowledge base: TRUE, FALSE or UNKNOWN.
62	if verdict is Verdict.UNKNOWN:	If the KB cannot say, hand over to the unknown-claim rule (next table) and stop.
63	self._handle_unknown(sim, view, r)	
64	return	
65	self._record(view.source, verdict is Verdict.FALSE)	Otherwise record the verdict in the source's credibility ledger.
66	if verdict is Verdict.FALSE:	A FALSE verdict quarantines the claim, halting its spread. A TRUE verdict needs no action.
67	sim.suppress(view.id, ClaimStatus.QUARANTINED, r)	

This next function is **where λ acts**. The knowledge base is sound, so a verified verdict never risks a false positive. Only the claims it cannot resolve involve a real trade-off.

_handle_unknown(): the λ trade-off

src/infodemic/agents/moderator.py, starting at line 69

69	def _handle_unknown(self, sim: Simulation, view: ClaimView, r: int) -> None:	The KB answered UNKNOWN. What now?
70	cfg = self.cfg	Shorthand for the moderator's settings.
71	if cfg.unknown_policy == "adaptive":	The default rule is adaptive.
72	harm = view.p_false * view.expected_next * cfg.lookahead	Benefit of acting. Estimated harm = chance it is false x expected new exposures next hop x a lookahead of 3 hops.
73	limit = harm * (1 - cfg.rate_limit_factor) > cfg.lam * (1 - view.p_false)	The λ rule. Rate-limit if the exposure prevented, after allowing for the 30% that still leaks through, exceeds λ times the chance the claim is true. High λ makes the right side large, so the moderator acts less.
74	escalate = harm >= cfg.escalate_threshold	Escalate to human review if the potential harm is at least 1 exposure.
75	else:	The fixed alternatives, kept for comparison...
76	limit = cfg.unknown_policy == "rate_limit"	...are simply: always rate-limit, always escalate, or always ignore.
77	escalate = cfg.unknown_policy == "escalate"	
78	if limit:	Provisional rate-limit...
79	sim.suppress(view.id, ClaimStatus.RATE_LIMITED, r)	...applied here. It counts as a takedown, and as a false positive if the claim is actually true.
80	if escalate and len(self.reviews) < cfg.escalation_capacity:	Escalate only if there is spare review capacity (2 at a time)...
81	self.reviews.append((r + cfg.escalation_delay, view.id))	...and the review is due 3 rounds from now.

_resolve_reviews(): slow, accurate human review

src/infodemic/agents/moderator.py, starting at line 83

83	def _resolve_reviews(self, sim: Simulation, r: int) -> None:	Finish escalated reviews that have reached their due round.
84	due = [cid for when, cid in self.reviews if when <= r]	The claims whose review is due now.
85	self.reviews = [(when, cid) for when, cid in self.reviews if when > r]	The rest keep waiting.
86	for cid in due:	For each due review...
87	is_false = sim.human_review(cid, self.cfg.escalation_accuracy)	...ask the simulator's human reviewer. It is right 95% of the time and wrong 5%, which is the source of the false-positive floor at high λ .
88	self._record(sim.view(cid).source, is_false)	Update the credibility ledger with the outcome.
89	if is_false:	If the reviewer says false, quarantine the claim.
90	sim.suppress(cid, ClaimStatus.QUARANTINED, r)	
91	else:	Otherwise lift any rate-limit. The claim already counted as a false positive if it was true, and that stays.
92	sim.release(cid)	

5.9 The simulation loop: env/simulation.py

Simulation.__init__(): build one world

src/infodemic/env/simulation.py, starting at line 37

37 def __init__(self, cfg: SimConfig, spreader_policy: str, moderator_policy: str, seed: int, shock: bool = False):	One episode: a config, the two strategy names, a seed, and whether the shock is on.
38 seeds = np.random.SeedSequence(seed).spawn(len(STREAMS)) 39 self.rngs = {name: np.random.default_rng(s) for name, s in zip(STREAMS, seeds)}	Spawn eight independent random streams from the one seed and name them. This is what makes comparisons paired.
40 self.cfg = cfg 41 self.shock = shock	Keep the config, and whether this is a shock run.
42 self.net = build_network(cfg.graph, self.rngs["graph"])	Build the graph from the graph stream.
43 world = generate_world(self.rngs["world"])	Build the synthetic world from the world stream.
44 kb = KnowledgeBase(sample_kb_facts(world, cfg.moderator.kb_coverage, self.rngs["world"]), RULES)	Give the moderator a KB holding a random kb_coverage (70%) of the world's base facts, plus the rules.
45 self.factory = ClaimFactory(world, cfg.claims.evidence_noise)	The factory that creates claims.
46 seeding = make_seeding_policy(spreader_policy, cfg.spreader)	Turn the spreader strategy name into a seeding policy.
47 self.spreader = SpreaderAgent(cfg.spreader, seeding, self.rngs["spreader"])	The main spreader, using its own stream.
48 self.swarm = BotSwarm(cfg.shock, cfg.spreader, seeding, self.rngs["swarm"]) if shock else None	The bot swarm exists only in shock runs.
49 self.moderator = Moderator(cfg.moderator, kb, make_moderator_policy(moderator_policy), self.rngs["moderator"])	The moderator, with the KB and the chosen selection policy.
50 self.claims: dict[int, Claim] = {} 51 self.round = 0 52 self.stats: list[RoundStats] = [] 53 self.takedowns = 0 54 self.false_positives = 0 55 self._created: dict[int, list[int]] = defaultdict(list) 56 self._suppressions: list[tuple[int, int]] = [] 57 self._incidence = 0	Bookkeeping: all claims, the current round, per-round stats, running counts of takedowns and false positives, which claims were created in which round, the public suppression log, and the current round's new-exposure count.

Simulation.step(): the seven steps of a round

src/infodemic/env/simulation.py, starting at line 59

59 def step(self) -> None:	Advance the simulation by one round.
60 r = self.round	The round number.
61 self._incidence = 0	Reset this round's count of new false exposures.
62 self.spreader.act(self, r)	1. The main spreader acts.
63 if self.swarm: 64 self.swarm.act(self, r)	2. In shock runs, the swarm acts too.
65 self._post_organic(r)	3. Organic claims arrive.
66 self.moderator.act(self, r)	4. The moderator acts.
67 for claim in self.claims.values(): 68 new = cascade_step(self.net, claim, self.rngs["cascade"], self.cfg.moderator.rate_limit_factor)	5. Every claim spreads one Independent Cascade hop.
69 if claim.is_false: 70 self._incidence += new	Only false claims add to the infection curve.
71 self.stats.append(self._snapshot(r))	6. Store this round's stats.
72 self.round += 1	7. Advance the clock.

post_claim()

src/infodemic/env/simulation.py, starting at line 80

80	def post_claim(self, source: int, is_false: bool, origin: str, rng: np.random.Generator) -> Claim:	Create and seed a claim. Both agents and the organic stream use this.
81	claim = self.factory.make(source, self.round, is_false, origin, rng)	Make the claim, drawing its atom and noisy content score from the caller's own random stream.
82	claim.exposed.add(source)	The source user is exposed at once and is the first carrier.
83	claim.frontier.append(source)	
84	self.claims[claim.id] = claim	Register the claim.
85	self._created[self.round].append(claim.id)	Remember it was created this round, so the moderator flags it after the detection delay.
86	if is_false and not self.net.is_bot(source):	A false claim seeded at a real user counts as one exposure right away. Sockpuppets do not count.
87	self._incidence += 1	
88	return claim	Hand the claim back.

suppress(): takedowns and false positives are counted here

src/infodemic/env/simulation.py, starting at line 104

104	def suppress(self, claim_id: int, status: ClaimStatus, r: int) -> None:	Rate-limit or quarantine a claim.
105	claim = self.claims[claim_id]	Look it up.
106	if claim.status is ClaimStatus.QUARANTINED or claim.status is status:	Ignore repeats: a quarantined claim cannot be suppressed further, and asking for the same status again does nothing.
107	return	
108	if status is ClaimStatus.QUARANTINED:	If this is a quarantine...
109	claim.blocked = set(claim.frontier)	...the users who were about to pass the claim on are blocked, and the frontier is emptied, so it cannot spread.
110	claim.frontier = []	
111	claim.status = status	Store the new status.
112	if not claim.takedown_counted:	The first time this claim is suppressed...
113	claim.takedown_counted = True	...mark it as counted...
114	self.takedowns += 1	...count one takedown...
115	self.false_positives += not claim.is_false	...and one false positive if the claim is actually true. A claim counts once even if it is rate-limited and then quarantined.
116	self._suppressions.append((r, claim_id))	Publish it in the public log. This is what the spreader watches.

5.10 The bot-swarm shock: scenarios/bot_swarm.py

BotSwarm.act()

src/infodemic/scenarios/bot_swarm.py, starting at line 26

26	def act(self, sim: Simulation, r: int) -> None:	Called every round; does nothing until the shock.
27	if r == self.shock.round:	At the scheduled round...
28	end = r + self.shock.duration	...the swarm lasts duration rounds (3).
29	self.agents = [	...create num_spreaders (12) new spreader agents, each with a window of exactly those rounds. They use the same targeting policy as the main spreader and are not adaptive, so no sockpuppets.
30	SpreaderAgent(self.spreader, self.policy, self.	
31	rng, start=r, end=end, adaptive=False)	
32	for _ in range(self.shock.num_spreaders)	
33	]	
33	for agent in self.agents:	Every round, each active swarm agent posts its 3 claims. That is 36 extra false claims per round for 3 rounds, against a moderator that can check 2.
34	agent.act(sim, r)	

5.11 The metrics: metrics.py

_settling_time(): how containment and recovery are defined

src/infodemic/metrics.py, starting at line 38

38 def _settling_time(curve: np.ndarray, threshold: int) -> tuple[int, bool]:	Given the curve, find when it settles below a threshold.
39 """Rounds until the curve stays at or below threshold, and whether it does so by the end."""	The docstring: rounds until it stays at or below the threshold, and whether that happens by the end.
40 above = np.flatnonzero(curve > threshold)	The rounds where the curve is above the threshold.
41 if len(above) == 0:	Never above: it settled at once, so 0 rounds.
42 return 0, True	
43 last = int(above[-1])	The last round above the threshold.
44 return last + 1, last < len(curve) - 1	It settles one round later. If that last round was the final one, it never settled, so the flag is False and the value is capped at the horizon.

compute_metrics()

src/infodemic/metrics.py, starting at line 47

47 def compute_metrics(stats: list[RoundStats], threshold: int, shock_round: int None = None) -> RunMetrics:	Turn the per-round records into one run's metrics.
48 """The infection curve is the number of users newly exposed to false claims each round."""	The infection curve is the count of users newly exposed to false claims each round.
49 curve = np.array([s.new_false_exposures for s in stats])	Turn the round records into an array.
50 ttc, contained = _settling_time(curve, threshold)	Time-to-containment: settle the whole curve, and say whether it did.
51 recovery = recovered = None	Recovery defaults to none, for runs without a shock.
52 if shock_round is not None:	
53 recovery, recovered = _settling_time(curve[shock_round:], threshold)	In shock runs, settle the part of the curve from the shock round onward. That is the recovery time.
54 takedowns, false_positives = stats[-1].takedowns, stats[-1].false_positives	Takedowns and false positives come from the last round's running totals. The RunMetrics returned after that also holds total exposure (the curve's sum) and peak (its maximum).

paired_difference(): the statistics behind every H1, H2 and H3 claim

src/infodemic/metrics.py, starting at line 68

68 def paired_difference(a: np.ndarray, b: np.ndarray, rng: np.random.Generator, n_boot: int = 5000) -> tuple[float, float, float]:	Compare two arrays of results that are paired by seed, returning the mean difference and a 95% confidence interval.
71 """Mean of a - b with a 95% bootstrap confidence interval; a and b are paired by seed."""	The docstring.
72 diffs = np.asarray(a, dtype=float) - np.asarray(b, dtype=float)	One difference per seed: same graph, same knowledge base, same organic claims, different strategy.
73 boots = rng.choice(diffs, size=(n_boot, len(diffs)), replace=True).mean(axis=1)	Bootstrap: resample those differences with replacement 5,000 times and take the mean each time.
74 low, high = np.percentile(boots, [2.5, 97.5])	The 2.5th and 97.5th percentiles of those means form the 95% interval.
75 return float(diffs.mean()), float(low), float(high)	Return the mean and the interval. A hypothesis counts as supported when the lower bound is above zero.

5.12 The experiment runners: experiments/

- `common.py`. `collect()` loops over every (condition, seed) pair, calls `run_simulation`, and returns two tables: one row per run with all metrics, and one row per run and round with the infection curve. It also holds the chart palette, the shared chart style and the command-line arguments every runner uses.
- `run_matrix.py`. Runs the six conditions without a shock. `test_hypotheses()` pairs runs by seed and applies `paired_difference` for H1 (centrality minus random targeting, per moderator) and H2 (FIFO or random minus impact-weighted, per spreader). It writes the summary table and draws the grouped bar chart.
- `run_shock.py`. Runs the same conditions with the shock and compares recovery times, including how many runs recover within $k = 15$ rounds. Draws the incidence curves around the shock.
- `run_lambda_sweep.py`. Repeats one condition (C6 by default) for nine values of λ from 0 to 200 and plots exposure against false positives.
- `run_sensitivity.py`. Repeats all six conditions while varying the budget, the KB coverage or the sockpuppet cap, and reports whether H2 still holds at each setting.

6. The outputs, and how to read them

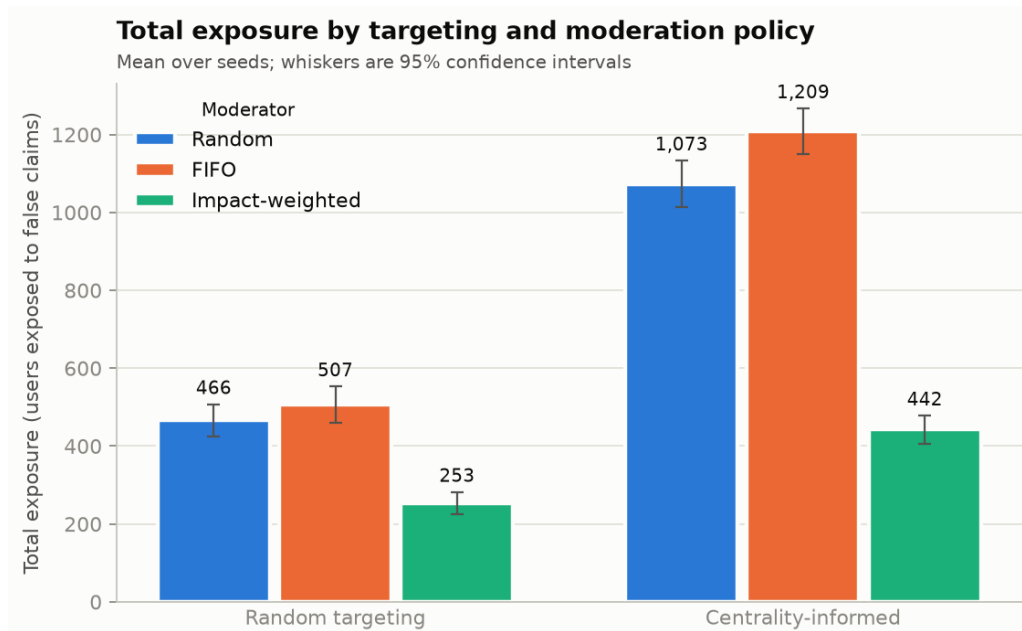
Every runner writes into `results/`. There are two kinds of files: **charts** (PNG) and **tables** (CSV). Both charts and the summary tables are committed to git because the README embeds them. The six raw per-run CSVs are gitignored: they regenerate identically because the seeds are fixed.

6.1 The files

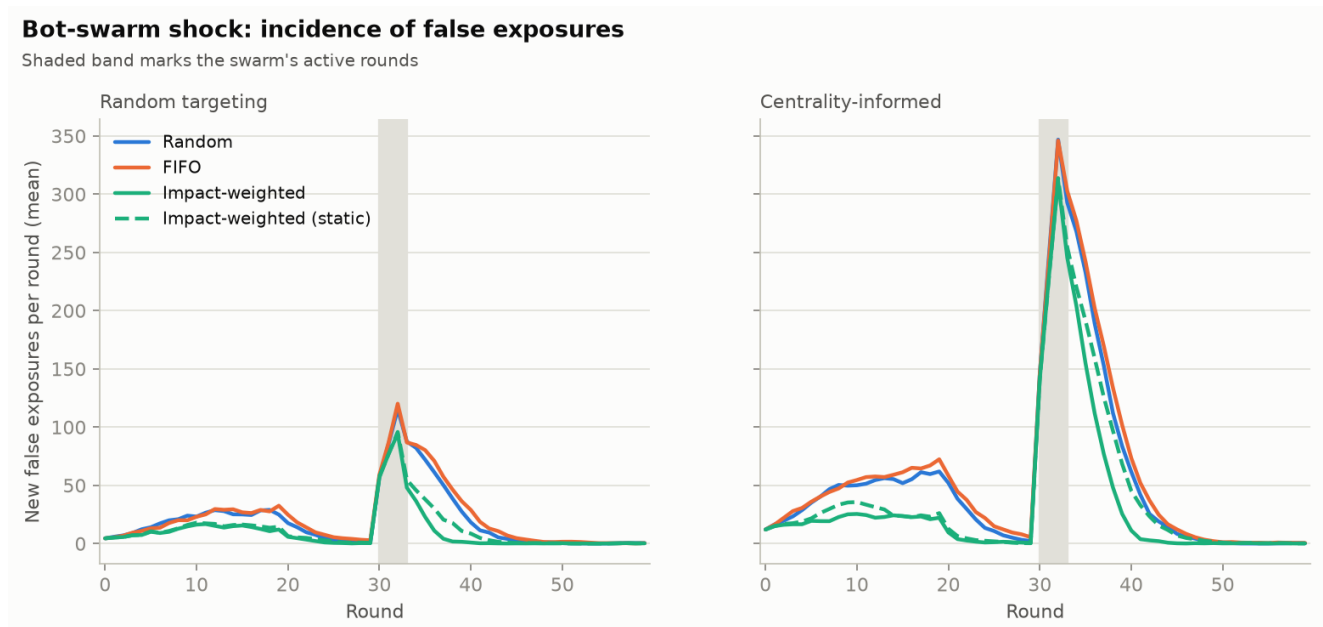
File	Made by	What is in it
<code>matrix_exposure.png</code>	<code>run_matrix.py</code>	Bar chart: mean total exposure for each spreader and moderator, with 95% intervals
<code>matrix_summary.csv</code>	<code>run_matrix.py</code>	One row per condition: mean and standard deviation of total exposure, peak, time-to-containment, false positives and false-positive rate
<code>matrix_hypotheses.csv</code>	<code>run_matrix.py</code>	H1 and H2 tests: mean difference, 95% interval, and whether the lower bound is above zero (supported)
<code>shock_recovery.png</code>	<code>run_shock.py</code>	Lines: mean new false exposures per round for each moderator, with the shock window shaded
<code>shock_summary.csv</code>	<code>run_shock.py</code>	Per condition: recovery time, fraction recovered, total exposure, peak, false positives, and the fraction recovered within k rounds
<code>shock_hypotheses.csv</code>	<code>run_shock.py</code>	H3 tests: how many rounds slower each static policy recovers than the impact-weighted one
<code>lambda_frontier.png</code>	<code>run_lambda_sweep.py</code>	Exposure against false positives, one point per λ
<code>lambda_sweep.csv</code>	<code>run_lambda_sweep.py</code>	For each λ : exposure, false positives, takedowns, false-positive rate, time-to-containment
<code>sensitivity_*.png</code> (3)	<code>run_sensitivity.py</code>	Exposure against the budget, the KB coverage and the sockpuppet cap
<code>sensitivity_h2.csv</code>	<code>run_sensitivity.py</code>	For every setting and spreader: is impact-weighted better than FIFO and random?
<code>*_runs.csv</code> (6, gitignored)	all runners	One row per single simulation with every metric. The raw material for all the summaries

Columns in a raw run file: `condition`, `spreader`, `moderator`, `seed`, `shock`, `total_exposure`, `peak_infection`, `time_to_containment`, `contained`, `false_positives`, `takedowns`, `false_positive_rate`, `recovery_time`, `recovered`, plus `recovered_within_k` in shock runs.

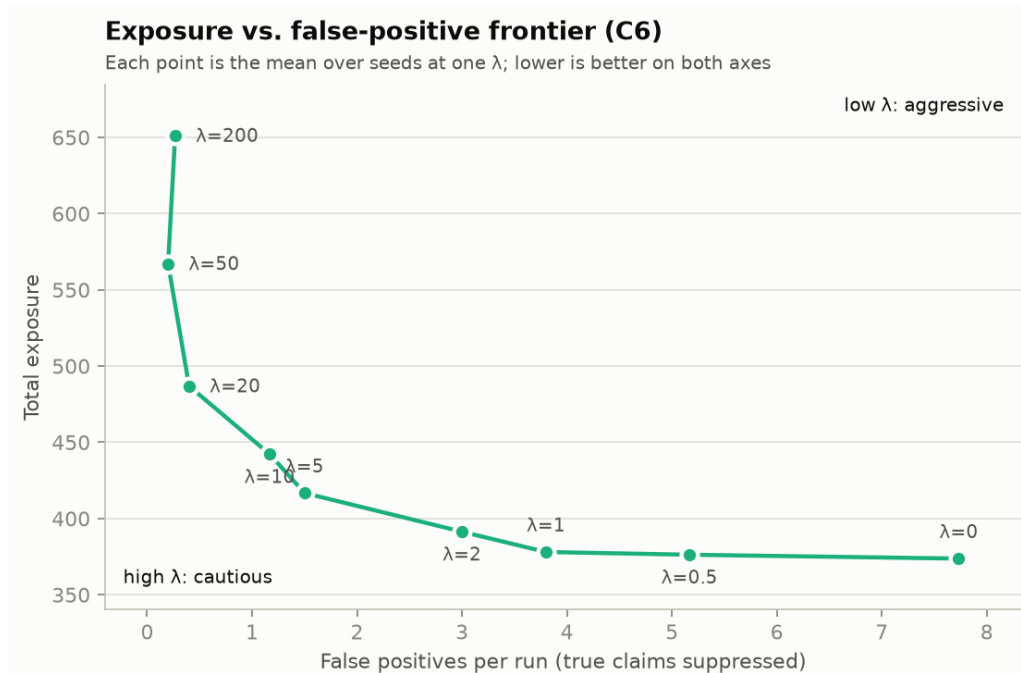
6.2 Reading the charts



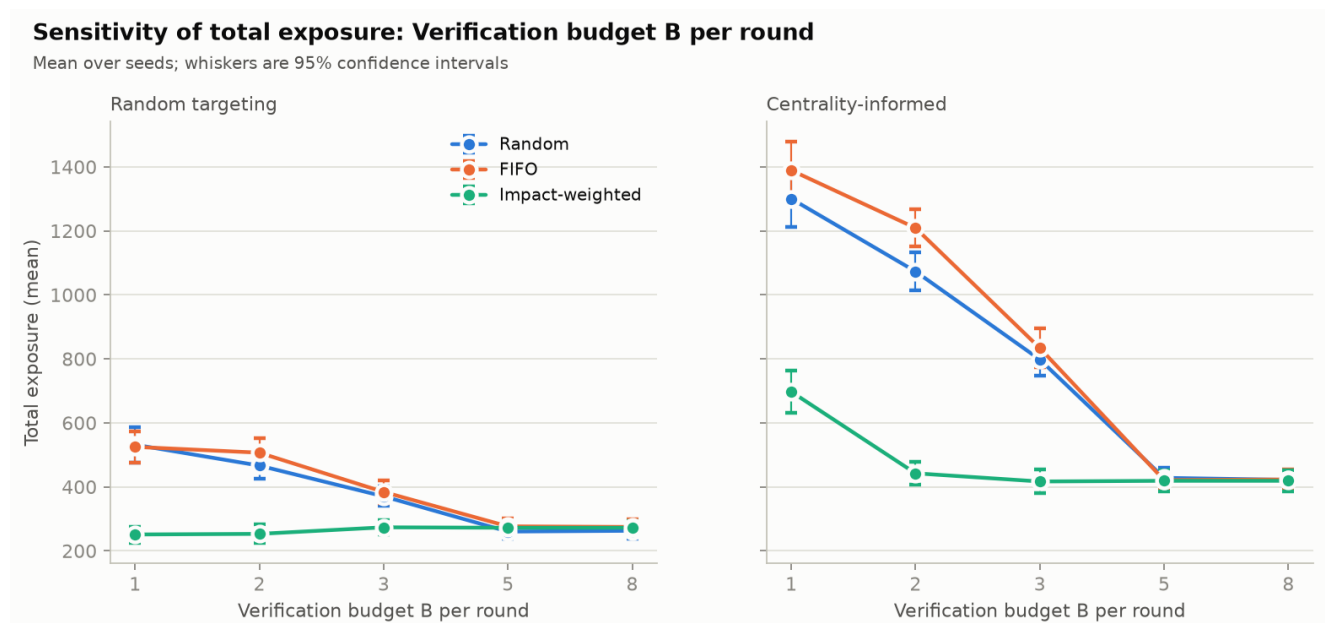
Matrix chart. Each cluster is a spreader strategy; each bar is a moderator. Lower is better for the moderator. The whiskers are 95% confidence intervals over 30 seeds. Look for two things: bars within a cluster (does the moderator matter?) and the same-colored bars across clusters (does hub targeting help the spreader?).



Shock chart. Each line is the average number of users newly exposed to false claims per round. The hump on the left is the spreader's normal campaign (rounds 0 to 19). The grey band is the bot swarm (rounds 30 to 32). A line that falls back to zero sooner is a faster recovery.



λ chart. Reading right to left, λ rises: the moderator gets more cautious, so false positives fall but exposure rises. The steep left side and the flat right side show that most of the benefit of aggression is captured by moderate λ .



Budget chart. Impact-weighted (green) is far below FIFO and random while the budget is small, but the three lines meet at a budget of 5 and above. Prioritization only matters when checks are scarce.

6.3 Reading a hypothesis row

A row such as centrality - random targeting | fifo moderator, mean_diff 702, ci 643 to 762, supported True means: across 30 paired seeds, the centrality spreader exposed on average 702 more users than the random spreader against a FIFO moderator, and the 95% bootstrap interval for that gap runs from 643 to 762. Since the whole interval is above zero, the difference is treated as real. supported is simply $ci_low > 0$.

7. The results

All numbers below are read directly from the CSV files in results/ at the time this document was built. Default config, 30 seeds per condition.

7.1 H1 and H2: informed play versus uninformed play

Spreader \ Moderator	Random	FIFO	Impact-weighted
Random targeting	C1: 466	C2: 507	C3: 253
Centrality-informed	C4: 1,073	C5: 1,209	C6: 442

Mean total exposure (users exposed to false claims). Lower is better for the moderator.

Hypothesis	Comparison (paired by seed)	Mean difference	95% interval	Supported
H1	centrality – random targeting , random moderator	+607	[554, 663]	Yes
H1	centrality – random targeting , fifo moderator	+702	[643, 762]	Yes
H1	centrality – random targeting , impact_weighted moderator	+189	[152, 229]	Yes
H2	fifo – impact_weighted , random spreader	+254	[202, 308]	Yes
H2	random – impact_weighted , random spreader	+213	[168, 266]	Yes
H2	fifo – impact_only , random spreader	+223	[170, 279]	Yes
H2	fifo – impact_weighted , centrality spreader	+767	[707, 825]	Yes
H2	random – impact_weighted , centrality spreader	+631	[580, 683]	Yes
H2	fifo – impact_only , centrality spreader	+766	[699, 830]	Yes

- **H1 is supported against every moderator.** Hub targeting gives the spreader roughly 600 to 700 extra exposures against random and FIFO moderators, and 189 against the impact-weighted one. The impact-weighted moderator cuts the hub advantage by roughly 70%.
- **H2 is supported against both baselines,** for both spreaders. Impact-weighted roughly halves exposure against the random spreader and cuts it by about 60% against the centrality spreader.
- **H2 does not depend on my suspicion weighting.** The `impact_only` ablation (velocity x reach alone, as the problem statement words it) gives almost the same numbers (rows C3p and C6p in `matrix_summary.csv`) and still beats FIFO.

7.2 H3: the bot-swarm shock

Condition (spreader / moderator)	Total exposure	Mean recovery (rounds)	Recovered within 15 rounds
C1 random / random	1,195	17.8	43%
C2 random / FIFO	1,321	18.1	37%
C3 random / impact-weighted	606	10.8	83%
C4 centrality / random	3,321	18.9	20%
C5 centrality / FIFO	3,593	20.3	7%
C6 centrality / impact-weighted	2,010	13.3	83%

Recovery gap (rounds slower than impact-weighted)	Mean	95% interval	Supported
fifo - impact_weighted , random spreader	7.3	[4.5, 9.9]	Yes
random - impact_weighted , random spreader	7.0	[4.3, 9.7]	Yes
impact_static (frozen ranking) - impact_weighted , random spreader	2.6	[0.1, 5.0]	Yes
fifo - impact_weighted , centrality spreader	7.1	[5.0, 9.1]	Yes

Recovery gap (rounds slower than impact-weighted)	Mean	95% interval	Supported
random - impact_weighted , centrality spreader	5.6	[3.6, 7.9]	Yes
impact_static (frozen ranking) - impact_weighted , centrality spreader	5.4	[3.3, 7.7]	Yes

- **H3 is supported as speed, not as a yes or no.** The impact-weighted moderator recovers 5.6 to 7.3 rounds faster than FIFO and random. But 93% or more of all runs eventually recover, so the claim that a static policy "does not" recover holds only in the sense of recovering much later.
- **Re-ranking under load is what helps.** In the frozen-ranking ablation, the score is fixed when a claim is first seen. Against the centrality spreader that ablation recovers about as slowly as FIFO and random (18.7 rounds against 20.3 and 18.9).
- **One borderline result.** Against the random spreader the frozen-ranking gap is 2.6 rounds with an interval of [0.07, 5.0], which only just excludes zero.

7.3 RQ3: the λ trade-off

λ	Total exposure	False positives per run	False-positive rate
0	374	7.73	12.9%
0.5	376	5.17	9.0%
1	378	3.80	6.8%
2	391	3.00	5.5%
5	417	1.50	2.8%
10	442	1.17	2.3%
20	487	0.40	0.8%
50	567	0.20	0.4%
200	651	0.27	0.6%

Condition C6 (centrality spreader, impact-weighted moderator), no shock.

- **Steep on the cautious side, flat on the aggressive side.** Lowering λ from 200 to 10 cuts exposure by about 209 for roughly 0.9 extra false positives per run. Going from 5 to 0 saves only about 43 more but adds about 6 false positives.
- **The frontier does not reach zero false positives.** About 0.27 per run remain at very high λ . That floor comes entirely from the 5% error rate of human review. With perfectly accurate review it drops to exactly 0.00.

7.4 Sensitivity: when does H2 stop holding?

run_sensitivity.py varies one setting at a time and re-tests H2 at each value. It holds in 24 of 28 settings. The four failures are all at a large verification budget.

Budget B	FIFO – impact, random spreader	FIFO – impact, centrality spreader	H2
1	+273.8	+691.9	Holds
2 (default)	+253.7	+766.8	Holds
3	+110.5	+417.0	Holds
5	+4.4	+2.3	Not supported
8	+2.5	+4.1	Not supported

- **Prioritization matters only while checks are scarce.** During the campaign about four claims arrive per round (three from the spreader, about one organic). Once B reaches 5, every policy can check nearly everything and they converge.
- **H2 holds at every KB coverage tested (30%, 50%, 70%, 90%) and every sockpuppet cap tested (0, 2, 5, 10, 20).**

- **The adaptive spreader is stronger.** Comparing 10 sockpuppets against 0, adaptation adds between 56 and 114 exposures depending on the condition.

One-line summary of the science. Informed play beats uninformed play on both sides: hub targeting helps the spreader, and impact-based prioritization helps the moderator, but only while verification is scarce. The moderator's λ then decides how much collateral damage it accepts to get there.

8. The tests

There are 43 automated tests in four files. They run in about two seconds with pytest. Several of them guard design intent, not just behavior: for example, that the moderator's view has no label, and that adapting never hurts the spreader.

File	Tests	What it checks
test_cascade.py	6	A certain reshare walks a path; a near-zero probability never spreads; each user gets only one chance; quarantine stops spread and blocks the frontier; rate-limiting scales the reshare probability (statistical, 400 trials); sockpuppets carry a claim but are never counted as exposed.
test_inference.py	13	Nine verdict cases (proven by a fact, proven by a rule, refuted by each negative rule, unknown); the neq built-in; recursive rules terminate; the KB is sound (over 500 true and 500 false claims it never gives a wrong verdict); partial coverage leaves claims unresolved.
test_policies.py	9	FIFO order; random selection respects the budget; impact ranks by velocity x reach; suspicion breaks ties; impact_only ignores suspicion; a static ranking ignores later changes while the adaptive one does not; seeding policies pick from the right pools; hubs have more than twice the degree of random picks; unknown policy names are rejected.
test_simulation.py	15	The same seed reproduces a run exactly; total exposure equals the users actually exposed to false claims; the moderator's view has no is_false field ; takedown accounting; the shock raises exposure and records recovery; sockpuppets appear only for an adaptive spreader; adaptation does not hurt the spreader ; a zero-budget moderator never acts; a lower λ takes down more true claims; metrics from a hand-made curve; uncontained runs are capped at the horizon; recovery is measured from the shock round; the paired bootstrap; the default config files load; invalid configs are rejected.

What is not unit-tested. The runner scripts and the chart code were checked by running them and looking at the output, not by automated tests. The whole quickstart was also run once from a clean copy in a brand-new virtual environment. Python 3.14 is the only version it has been run on.

9. Limitations and honest notes

- **No dataset, by design.** Claims are synthetic predicates, the graph is generated, and the knowledge base is small and hand-authored. Results describe decision strategies, not real fact-checking accuracy or real platform behavior.
- **Hand-set parameters.** Reshare probability, claim arrival rate, evidence noise, the weights that turn evidence into a probability of falsehood, and the moderator's lookahead are fixed choices. Only λ , the budget, KB coverage and the sockpuppet cap have been swept.
- **The budget choice matters.** $B = 2$ was chosen so that verification is scarce. The sensitivity analysis shows the main result (H2) disappears once B is 5 or more.
- **One graph family.** Every run uses a 750-node Barabási-Albert graph. Other sizes and real network structures have not been tried.
- **Simplified actions.** Counter-claims are not a separate action; quarantine stands in for them. The spreader never mixes in true claims to build credibility.
- **The evidence model is simple.** Content scores are drawn from normal distributions around 0.65 (false) and 0.40 (true), and the moderator's combination weights are set by hand rather than learned.
- **Definitions are mine.** The problem statement names the metrics but not their exact formulas. Total exposure is the sum of the infection curve; containment means new false exposures stay at or below 2

per round; H3's "within k rounds" uses $k = 15$. These choices are documented in the README.

- **Fixed seeds.** Seeds 0 to 29 are fixed, so every result is exactly reproducible. The confidence intervals reflect variation across those 30 graphs and claim streams.
- **H3's wording.** "A static policy does not recover" is not literally true here: 93% or more of runs recover. The supported claim is that the adaptive moderator recovers faster.
- **Not yet done.** The report/ folder is empty, and the slides have not been made.

10. Presenting the project

10.1 A suggested ten-slide outline

#	Slide	Message	Take it from
1	The problem	False stories outrun verification, and verification has a budget	Section 1
2	The two agents	A spreader and a moderator with opposed goals; ordinary users are passive	Section 1 cast table
3	The environment	A 750-node scale-free graph with hubs; Independent Cascade spread	Section 4.2
4	How the moderator verifies	Horn-clause rules; TRUE, FALSE or UNKNOWN; about 40% unresolved	Section 5.2
5	How it decides	Rank by impact; the λ rule for unresolved claims	Sections 5.7, 5.8
6	Experiment design	2 x 3 matrix, 30 paired seeds, bootstrap intervals	Sections 4.4, 7
7	H1 and H2	Hub targeting helps the spreader; impact ranking helps the moderator	Matrix chart
8	The shock and H3	Impact ranking recovers 5.6 to 7.3 rounds faster	Shock chart
9	λ and sensitivity	The trade-off frontier; H2 needs scarce verification	λ chart, budget chart
10	Limits and next steps	Synthetic by design; sweep more parameters; real graph structure	Section 9

10.2 Six numbers worth remembering

Number	Meaning
+607 and +702	Extra exposures a hub-targeting spreader causes against random and FIFO moderators (H1)
253 and 442	Exposure under the impact-weighted moderator, against 466 to 507 and 1,073 to 1,209 for the baselines (H2)
5.6 to 7.3 rounds	How much faster the impact-weighted moderator recovers from the shock (H3)
24 of 28	Sensitivity settings where H2 holds. All four failures are at a budget of 5 or more
374 to 651	Total exposure as λ rises from 0 to 200, while false positives fall from 7.7 to about 0.27 per run
about 40%	Share of claims the knowledge base cannot resolve

10.3 Questions to expect

Question	Short answer
Why is there no dataset?	The problem statement specifies synthetic predicates, a generated graph and a hand-authored KB. Real fake-news datasets are text-based and would need NLP, which is out of scope. A real social graph could be a future robustness check for the network structure.
Does the moderator cheat by knowing the truth?	No. The moderator sees a ClaimView that has no label, and a test enforces that. Truth reaches it only through a KB verdict or a human review.
Do the results depend on your parameters?	Partly, and we measured how. The budget matters most: H2 holds at B of 3 or less and disappears at 5 or more, because verification stops being scarce. It is robust to KB coverage and sockpuppet count.
What does the adaptive opponent actually do?	When its claims get suppressed it creates sockpuppets that reshare its strongest claim. That adds 56 to 114 exposures. The adaptation is a simple rule, not a learned strategy.
What is a false positive?	A true claim that was ever rate-limited or quarantined, counted once. It can only happen when the KB cannot resolve a claim or when human review is wrong.
Why do false positives not reach zero at high λ ?	Human review is 95% accurate. With perfect review, false positives at $\lambda = 200$ drop to exactly 0.00.
How do you know the gaps are not luck?	Every comparison is paired by seed (same graph, KB and organic claims), and each gap has a 95% bootstrap interval. A result counts only if the whole interval is above zero.
Why did an earlier version disagree with H1?	An earlier spreader design made the centrality spreader weaker when it adapted. It was found and fixed before presenting, and all results were regenerated.
What would you do next?	Sweep graph size and reshare probability, try a real graph topology, let the spreader mix in true claims, and replace hand-set weights with learned ones.

10.4 A short live demo

```
source .venv/bin/activate
python experiments/run_matrix.py --seeds 5 --out /tmp/demo # 1.8 seconds; prints the H1 and H2 tables
xdg-open /tmp/demo/matrix_exposure.png # opens the chart
```

Use `--seeds 5` for a quick demo, and **always add** `--out`: without it the run overwrites the committed charts in `results/` with 5-seed versions. The intervals will be wider than in the full 30-seed run.

Appendix A. File inventory

File	Lines	Purpose
src/infodemic/config.py	135	Typed configs, YAML loading, validation, override()
src/infodemic/metrics.py	75	RoundStats, RunMetrics, compute_metrics, paired_difference
src/infodemic/env/network.py	54	Scale-free graph, trust, hubs, sockpuppets
src/infodemic/env/claims.py	109	Claim, ClaimView, node and claim states, ClaimFactory
src/infodemic/env/cascade.py	29	One Independent Cascade hop
src/infodemic/env/simulation.py	146	The episode and round loop
src/infodemic/agents/spreader.py	73	Spreader agent with sockpuppet adaptation
src/infodemic/agents/moderator.py	92	Moderator agent: verify, act, escalate
src/infodemic/policies/spreader_policies.py	37	Random and centrality seeding
src/infodemic/policies/moderator_policies.py	65	Random, FIFO, impact-weighted and ablations
src/infodemic/kb/facts.py	76	Synthetic world, true and false atoms, KB sampling
src/infodemic/kb/rules.py	19	The six Horn rules
src/infodemic/kb/inference.py	109	Backward-chaining engine and verdicts
src/infodemic/scenarios/bot_swarm.py	34	The coordinated shock
experiments/common.py	113	Shared run collection, CLI arguments, chart style
experiments/run_matrix.py	105	The 2 x 3 matrix (H1, H2)
experiments/run_shock.py	98	Shock runs (H3)
experiments/run_lambda_sweep.py	88	The λ sweep (RQ3)
experiments/run_sensitivity.py	128	Sensitivity sweeps
experiments/configs/default.yaml	43	All default parameters
experiments/configs/conditions.yaml	7	Definition of C1 to C6
tests/test_cascade.py	65	6 tests
tests/test_inference.py	67	13 tests
tests/test_policies.py	82	9 tests
tests/test_simulation.py	123	15 tests
Total (files listed)	1972	Package __init__.py files, pyproject.toml, requirements.txt, .gitignore and the README are not counted

Appendix B. Configuration reference

Every value below is read from `experiments/configs/default.yaml`. Change them by copying that file and passing it with `--config`.

Parameter	Default	Meaning
<code>horizon</code>	60	Rounds per episode
<code>containment_threshold</code>	2	New false exposures per round at or below which spread counts as contained
<code>graph.num_nodes</code>	750	Real users in the network
<code>graph.attachment_edges</code>	3	Links each new node makes while the scale-free graph is built
<code>graph.base_reshare_prob</code>	0.1	Base probability a user passes a claim to a neighbor
<code>graph.trust_range</code>	[0.6, 1.4]	Range of per-user trust scores that multiply that probability
<code>graph.sockpuppet_trust</code>	1.0	Trust score of a sockpuppet account
<code>spreader.posts_per_round</code>	3	False claims posted per round
<code>spreader.campaign_rounds</code>	20	Rounds the main spreader posts (rounds 0 to 19)
<code>spreader.sockpuppets_per_round</code>	1	New sockpuppets per round when alarmed
<code>spreader.max_sockpuppets</code>	10	Cap on sockpuppets. 0 turns the adaptation off
<code>spreader.sockpuppet_links</code>	8	Accounts each sockpuppet links to
<code>spreader.hub_fraction</code>	0.05	Share of users, by degree, counted as hubs
<code>moderator.budget_per_round</code>	2	Claims verified per round (B)
<code>moderator.lam</code>	10.0	λ : the cost of one false positive, in units of exposure
<code>moderator.detection_delay</code>	1	Rounds between a claim's creation and its being flagged
<code>moderator.rate_limit_factor</code>	0.3	Multiplier on the reshare probability of a rate-limited claim
<code>moderator.kb_coverage</code>	0.7	Share of the world's base facts the knowledge base holds
<code>moderator.queue_ttl</code>	25	Rounds a claim stays in the queue
<code>moderator.unknown_policy</code>	adaptive	How UNKNOWN claims are handled: adaptive, rate_limit, escalate or ignore
<code>moderator.lookahead</code>	3.0	Hops of future exposure counted when estimating harm
<code>moderator.escalate_threshold</code>	1.0	Minimum estimated harm before escalating to human review
<code>moderator.escalation_capacity</code>	2	Human reviews in progress at once
<code>moderator.escalation_delay</code>	3	Rounds a human review takes
<code>moderator.escalation_accuracy</code>	0.95	Probability a human review is correct
<code>claims.organic_rate</code>	1.0	Mean number of organic claims per round
<code>claims.organic_false_prob</code>	0.1	Chance an organic claim is false
<code>claims.evidence_noise</code>	0.15	Standard deviation of a claim's noisy content score
<code>shock.round</code>	30	Round the bot swarm starts
<code>shock.num_spreaders</code>	12	Extra spreaders in the swarm
<code>shock.duration</code>	3	Rounds the swarm stays active
<code>seeds</code>	0 to 29 (30 seeds)	Random seeds used (0 to 29)